深度学习基础

第 3 讲 — 从机器学习到深度学习

孟翰

人工智能学院
中国石油大学（北京）

2026 年春季学期

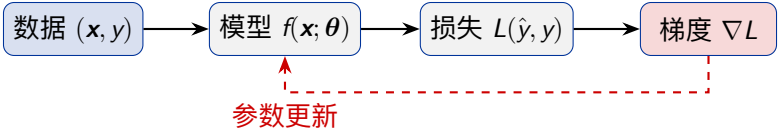
本讲内容

1. 机器学习回顾与线性模型的局限
2. 感知机与线性可分性
3. 从线性到非线性——多层网络
4. 前馈神经网络
5. 如何训练——反向传播
6. 过拟合、欠拟合与模型评估
7. 本讲小结与下讲预告

监督学习三要素

给定训练集 $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^N$，学习映射 $f: \boldsymbol{x} \to y$：

1. **模型**：定义函数族 $f(\boldsymbol{x}; \boldsymbol{\theta})$
2. **损失函数**：衡量预测与真实值的差距 $L(\hat{y}, y)$
3. **优化算法**：$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla_{\boldsymbol{\theta}} L$



本讲目标

从你们已经熟悉的线性模型出发，揭示其**根本局限**，自然引出深度学习的核心思想。

线性回归

$$\hat{y} = \boldsymbol{w}^\top \boldsymbol{x} + b$$

- 输出：连续实数值
- 损失：MSE
- 决策面：超平面

逻辑回归（二分类）

$$P(y = 1|\boldsymbol{x}) = \sigma(\boldsymbol{w}^\top \boldsymbol{x} + b)$$

- 输出：概率值 $(0, 1)$
- 损失：二元交叉熵
- 决策面：超平面

共同本质

无论如何变换输出形式，核心都是**线性函数** $\boldsymbol{w}^\top \boldsymbol{x} + b$——决策边界永远是超平面，**无法学习非线性关系**。

问题

二分类用 Sigmoid，那**多分类**（K 个类别）怎么办?

Softmax 函数

给定 logits $\boldsymbol{z} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b} \in \mathbb{R}^K$，Softmax 将其转化为概率分布：

$$\boxed{\text{softmax}(z_k) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \quad k = 1, 2, \ldots, K}$$

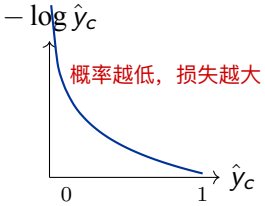
关键性质

- 输出在 $(0, 1)$，且 $\sum_{k=1}^{K} \hat{y}_k = 1$（合法概率分布）
- 指数函数**放大差异**：$\boldsymbol{z} = [2.0, 1.0, 0.1] \to \hat{\boldsymbol{y}} = [0.659, 0.242, 0.099]$
- 保持排序：$z_i > z_j \Rightarrow \hat{y}_i > \hat{y}_j$

多分类交叉熵

$$\boxed{L = -\sum_{k=1}^{K} y_k \log \hat{y}_k}$$

若正确类别为 c（$y_c = 1$），简化为 $L = -\log \hat{y}_c$



信息论解释

$$H(P, Q) = H(P) + D_{\mathrm{KL}}(P \| Q)$$

- $H(P)$：真实分布的熵（常数）
- D_{KL}：KL 散度（非负）
- 最小化交叉熵 $\Leftrightarrow$ 最小化 D_{KL}

Softmax + 交叉熵的联合梯度推导

设 $\hat{y}_k = \text{softmax}(z_k)$，$L = -\log \hat{y}_c$，求 $\frac{\partial L}{\partial z_k}$：

$k = c$（正确类别）

$$\frac{\partial L}{\partial z_c} = -\frac{1}{\hat{y}_c} \cdot \hat{y}_c(1 - \hat{y}_c) = \hat{y}_c - 1$$

$k \neq c$（非正确类别）

$$\frac{\partial L}{\partial z_k} = -\frac{1}{\hat{y}_c} \cdot (-\hat{y}_c \hat{y}_k) = \hat{y}_k$$

统一表达

$$\boxed{\frac{\partial L}{\partial \boldsymbol{z}} = \hat{\boldsymbol{y}} - \boldsymbol{y}}$$

梯度 = 预测概率 − 真实标签。**极其简洁**，这是 Softmax+ 交叉熵成为标准配置的数学原因。

练习

给定 $\boldsymbol{z} = [3.0,\ 1.0,\ -1.0]$，正确标签为第 1 类。求：(1) Softmax 概率 (2) 交叉熵损失 (3) 梯度

解答

1. $e^{3.0} = 20.09$，$e^{1.0} = 2.72$，$e^{-1.0} = 0.37$，和 $= 23.18 \Rightarrow \hat{\boldsymbol{y}} = [0.867,\ 0.117,\ 0.016]$
2. $L = -\log(0.867) = 0.143$
3. $\frac{\partial L}{\partial \boldsymbol{z}} = \hat{\boldsymbol{y}} - \boldsymbol{y} = [0.867-1,\ 0.117-0,\ 0.016-0] = [-0.133,\ 0.117,\ 0.016]$

思考

如果正确类别改为第 3 类？$L = -\log(0.016) = 4.14$，损失**大幅增加**！

三种线性模型的共同本质

- 线性回归：$\hat{y} = \boldsymbol{w}^\top \boldsymbol{x} + b$
- 逻辑回归：$\hat{y} = \sigma(\boldsymbol{w}^\top \boldsymbol{x} + b)$
- Softmax 回归：$\hat{\boldsymbol{y}} = \text{softmax}(\boldsymbol{W}\boldsymbol{x} + \boldsymbol{b})$

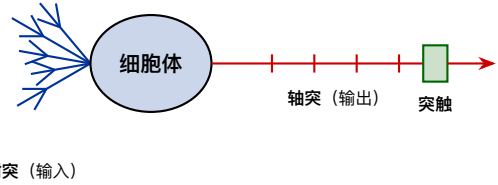
$\longrightarrow$ **决策边界都是超平面！**

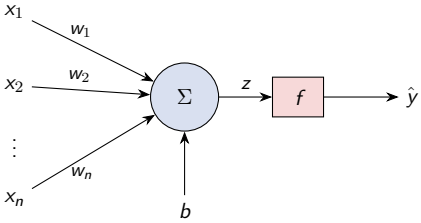
根本局限

1. **表达能力有限**：无法拟合非线性函数
2. **特征工程依赖**：需要人工设计非线性特征（多项式、核函数等）
3. **无法自动学习表示**：特征的好坏决定模型的上限

核心问题

能否设计一个模型，**自动**从数据中学习非线性特征表示？





生物原理

1. 树突接收电信号
2. 细胞体汇总信号
3. 超过阈值 → 通过轴突发射

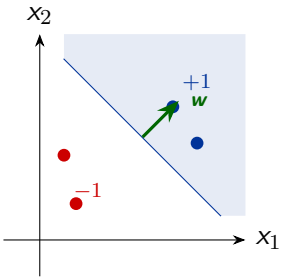
数学表达

$$z = \sum_{i=1}^{n} w_i x_i + b = \boldsymbol{w}^\top \boldsymbol{x} + b$$

$$\hat{y} = f(z)$$

Rosenblatt 感知机 (1957)

$$\hat{y} = \text{sign}(\boldsymbol{w}^\top \boldsymbol{x} + b) = \begin{cases} +1, & \boldsymbol{w}^\top \boldsymbol{x} + b \geq 0 \\ -1, & \boldsymbol{w}^\top \boldsymbol{x} + b < 0 \end{cases}$$



几何解释

- 超平面 $\boldsymbol{w}^\top \boldsymbol{x} + b = 0$ 分割空间
- $\boldsymbol{w}$：法向量；b：偏移距离
- 数据必须**线性可分**

学习规则

对误分类样本 $(\boldsymbol{x}, y)$：

$$\boldsymbol{w} \leftarrow \boldsymbol{w} + \eta y \boldsymbol{x}, \quad b \leftarrow b + \eta y$$

AND 门： $w_1 = 1, w_2 = 1, b = -1.5$

x_1	x_2	z	$\hat{y}$
0	0	-1.5	0
0	1	-0.5	0
1	0	-0.5	0
1	1	$+0.5$	1

OR 门： $w_1 = 1, w_2 = 1, b = -0.5$

x_1	x_2	z	$\hat{y}$
0	0	-0.5	0
0	1	$+0.5$	1
1	0	$+0.5$	1
1	1	$+1.5$	1

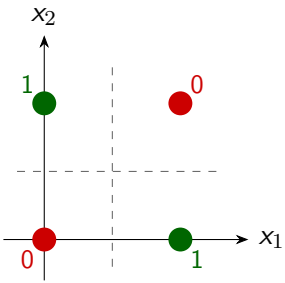
观察

AND 和 OR 都是**线性可分**的——可以画一条直线将 0 和 1 分开。

但是，有一个基本的逻辑运算感知机**无法实现**——XOR！

XOR 真值表

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0



无法线性分割！

直觉

相同输入对输出 0，不同输入对输出 1——对角线上的点属于同一类，无法用一条直线分开。

假设存在 w_1, w_2, b 使得感知机实现 XOR，需同时满足：

$$(0,0) \to 0: \quad b < 0 \qquad (0,1) \to 1: \quad w_2 + b \geq 0 \qquad (1,\ 2)$$
$$(1,0) \to 1: \quad w_1 + b \geq 0 \qquad (1,1) \to 0: \quad w_1 + w_2 + b < 0 \qquad (3,\ 4)$$

矛盾推导

由 (2) + (3)：$w_1 + w_2 + 2b \geq 0$。由 (4)：$w_1 + w_2 < -b$。
代入：$-b + 2b \geq 0$，即 $b \geq 0$。但这与 (1) $b < 0$ **矛盾**！ □

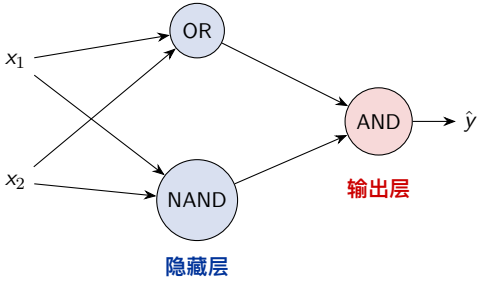
意义

这不仅是 XOR 的问题——**任何线性不可分**的数据都无法被单个感知机正确分类。这是 1969 年 Minsky/Papert 导致 AI 寒冬的核心论证。

关键洞察

XOR 不能用一条线分，但可以用**两条线的组合**来分！

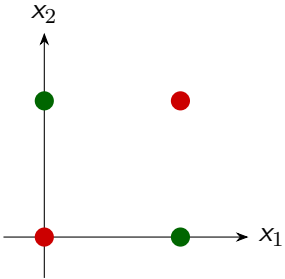
$\mathrm{XOR}(x_1, x_2) = \mathrm{AND}\big(\mathrm{OR}(x_1, x_2),\ \mathrm{NAND}(x_1, x_2)\big)$



验证：所有 4 种输入均正确

$(0,0) \to 0$✓ $(0,1) \to 1$✓ $(1,0) \to 1$✓ $(1,1) \to 0$✓

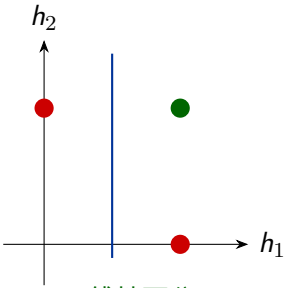
原始空间 (x_1, x_2)



不可线性分割

$\Rightarrow$
隐藏层
变换

特征空间 (h_1, h_2)



线性可分！

深度学习的核心思想

隐藏层将数据**非线性变换**到新的特征空间，使复杂问题变得**线性可分**。多层堆叠 = 逐层变换，从简单特征到复杂语义——这就是**表示学习**的本质。

线性层叠加定理

若每层都是**纯线性变换**（无激活函数），则多层网络等价于单层：

$$\boldsymbol{W}_2(\boldsymbol{W}_1\boldsymbol{x}+\boldsymbol{b}_1)+\boldsymbol{b}_2=\underbrace{\boldsymbol{W}_2\boldsymbol{W}_1}_{\boldsymbol{W}'}\boldsymbol{x}+\underbrace{(\boldsymbol{W}_2\boldsymbol{b}_1+\boldsymbol{b}_2)}_{\boldsymbol{b}'}=\boldsymbol{W}'\boldsymbol{x}+\boldsymbol{b}'$$

推广到任意层数

$$\boldsymbol{W}_L\boldsymbol{W}_{L-1}\cdots\boldsymbol{W}_1\boldsymbol{x}=\boldsymbol{W}'\boldsymbol{x}$$

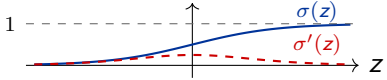
100 层线性网络 $\equiv$ 1 层线性网络！ 多层线性变换的复合仍然是线性变换。

结论

非线性激活函数是深度网络超越线性模型的**关键**。没有非线性，增加深度毫无意义。

定义与导数

$$\sigma(z) = \frac{1}{1+e^{-z}}, \quad \sigma'(z) = \sigma(z)(1-\sigma(z))$$



优点

- 输出有界 $(0,1)$，可解释为概率
- 平滑可微，导数形式优美

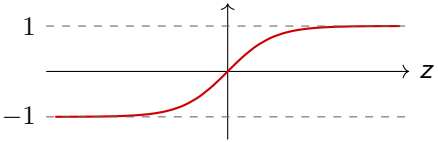
致命缺点

- **梯度消失**：$\sigma'(z)_{\max} = 0.25$
  10 层连乘：$0.25^{10} \approx 10^{-6}$
- **非零中心**：输出均值偏正
- 指数运算计算量较大

定义与导数

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = 2\sigma(2z) - 1$$

$$\tanh'(z) = 1 - \tanh^2(z)$$



相比 Sigmoid 的改进

- 输出范围 $(-1, 1)$，**零中心**
- 最大导数值为 1（$z = 0$ 处）
- 梯度消失问题**略有缓解**

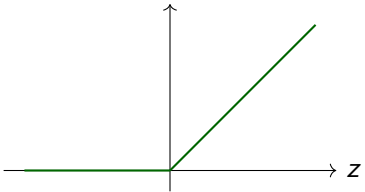
仍存在的问题

- 饱和区（$|z|$ 大）梯度仍趋近于 0
- 深层网络中梯度消失问题仍然严重
- 计算包含指数运算

定义与导数

$$\mathrm{ReLU}(z) = \max(0, z) = \begin{cases} z, & z > 0 \\ 0, & z \le 0 \end{cases}$$

$$\mathrm{ReLU}'(z) = \begin{cases} 1, & z > 0 \\ 0, & z \le 0 \end{cases}$$



为什么 ReLU 是革命性的?

- 正区间梯度恒为 **1**：彻底解决梯度消失
- 计算极简：$\max(0, z)$，无指数运算
- 稀疏激活：约 50% 神经元输出为 0
- 收敛速度比 Sigmoid 快 **6 倍**

缺点："死亡 ReLU"

- 负区间梯度为 0
- 若输入始终为负，该神经元**永远不更新**
- 大学习率可能导致大量神经元"死亡"

Leaky ReLU

$$f(z)=\begin{cases} z, & z>0 \\ \alpha z, & z\le 0 \end{cases}$$

$\alpha=0.01$，保留微小梯度

PReLU

$$f(z)=\begin{cases} z, & z>0 \\ \alpha z, & z\le 0 \end{cases}$$

α 为**可学习参数**

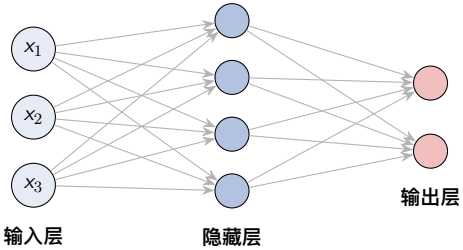
ELU

$$f(z)=\begin{cases} z, & z>0 \\ \alpha(e^z-1), & z\le 0 \end{cases}$$

平滑连续，均值 ≈ 0

激活函数选择指南

函数	输出范围	零中心	梯度消失	推荐场景
Sigmoid	$(0,1)$	否	严重	输出层（二分类）
Tanh	$(-1,1)$	是	严重	RNN 隐藏层
ReLU	$[0,+\infty)$	否	正区间无	**隐藏层默认**
Leaky ReLU	$(-\infty,+\infty)$	否	无	避免死亡神经元



前向传播公式

每层执行两步：$\boldsymbol{z}^{(l)} = \boldsymbol{W}^{(l)}\boldsymbol{a}^{(l-1)} + \boldsymbol{b}^{(l)}$，$\boldsymbol{a}^{(l)} = f(\boldsymbol{z}^{(l)})$。其中 $\boldsymbol{a}^{(0)} = \boldsymbol{x}$，$\boldsymbol{a}^{(L)} = \hat{\boldsymbol{y}}$。

关键要素

非线性激活函数 f 必不可少（否则多层退化为单层）。参数 $\{\boldsymbol{W}^{(l)}, \boldsymbol{b}^{(l)}\}_{l=1}^{L}$ 通过**训练**自动学习——但如何训练?

两层网络：输入 2 维，隐藏层 3 个 ReLU 神经元，输出 1 维，$\boldsymbol{x} = [1, 2]^\top$

第一层（隐藏层）：

$$\boldsymbol{W}^{(1)} = \begin{bmatrix} 0.2 & 0.4 \\ -0.5 & 0.1 \\ 0.3 & -0.2 \end{bmatrix}, \ \boldsymbol{b}^{(1)} = \begin{bmatrix} 0.1 \\ 0.2 \\ -0.1 \end{bmatrix} \quad \Rightarrow \quad \boldsymbol{z}^{(1)} = \begin{bmatrix} 1.1 \\ -0.1 \\ -0.2 \end{bmatrix}, \ \boldsymbol{a}^{(1)} = \mathrm{ReLU}(\boldsymbol{z}^{(1)}) = \begin{bmatrix} 1.1 \\ 0 \\ 0 \end{bmatrix}$$

第二层（输出层）：$\boldsymbol{W}^{(2)} = [0.7, \ -0.3, \ 0.5]$，$b^{(2)} = 0.1$

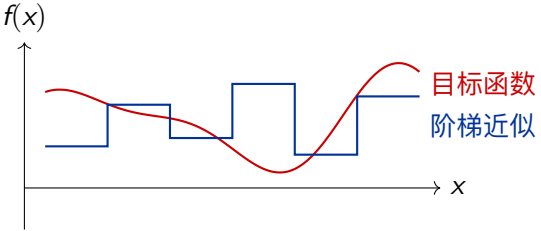
$$\hat{y} = 0.7 \times 1.1 + (-0.3) \times 0 + 0.5 \times 0 + 0.1 = 0.87$$

观察

ReLU 将负值“关闭”（$-0.1 \to 0, -0.2 \to 0$），只有激活的神经元参与后续计算——**稀疏性**。

定理 (Cybenko 1989, Hornik 1991)

具有**足够宽**隐藏层和非线性激活的前馈网络，可以**任意精度**近似紧集上的任意连续函数。



重要限定——定理不保证

1. **不保证可学习性**：存在 $\neq$ 梯度下降能找到
2. **不告诉你需要多宽**：可能需要指数级神经元
3. **不保证泛化**：拟合训练集 $\neq$ 新数据上表现好

线性模型（线性回归、Softmax 回归）
只能处理线性关系

局限：线性

↓

感知机
引入神经元概念，但仍是线性分类器

局限：XOR

↓

多层感知机 + 非线性激活
突破线性限制，理论上可逼近任意函数

问题：如何训练？

↓

深度网络
通过增加深度提高表达效率

答案：反向传播！

优化问题

给定训练集和网络结构，寻找最优参数 $\boldsymbol{\theta}^* = \{\boldsymbol{W}^{(l)}, \boldsymbol{b}^{(l)}\}_{l=1}^{L}$：

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \frac{1}{N} \sum_{i=1}^{N} L\big(f(\boldsymbol{x}_i; \boldsymbol{\theta}),\ y_i\big)$$

梯度下降需要什么？

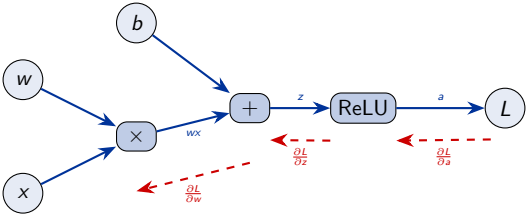
$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla_{\boldsymbol{\theta}} L$$

需要计算损失函数对**每个参数**的偏导数。一个简单的两层网络就有数十万参数！

关键问题

如何**高效**地计算所有参数的梯度？

答案：反向传播算法（Backpropagation, 1986, Rumelhart, Hinton & Williams）



反向传播的核心思想

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w} = \underbrace{\frac{\partial L}{\partial a} \cdot f'(z)}_{\text{上游梯度}} \cdot \underbrace{x}_{\text{本地梯度}}$$

- **前向**（蓝色实线）：从输入到损失，计算并保存中间值
- **反向**（红色虚线）：从损失到参数，逐层传递梯度

误差信号

$\boldsymbol{\delta}^{(l)} \triangleq \frac{\partial L}{\partial \boldsymbol{z}^{(l)}}$：第 l 层线性输出对最终损失的“责任”。

四个核心公式

1. **输出层误差**：$\boldsymbol{\delta}^{(L)} = \hat{\boldsymbol{y}} - \boldsymbol{y}$ （Softmax+CE 或线性 +MSE）
2. **误差递推**：$\boldsymbol{\delta}^{(l)} = \left((\boldsymbol{W}^{(l+1)})^\top \boldsymbol{\delta}^{(l+1)}\right) \odot f'(\boldsymbol{z}^{(l)})$
3. **权重梯度**：$\frac{\partial L}{\partial \boldsymbol{W}^{(l)}} = \boldsymbol{\delta}^{(l)} (\boldsymbol{a}^{(l-1)})^\top$
4. **偏置梯度**：$\frac{\partial L}{\partial \boldsymbol{b}^{(l)}} = \boldsymbol{\delta}^{(l)}$

直觉

误差通过 $\boldsymbol{W}^\top$ **逐层传回**，再被激活函数导数 f' 缩放。

目标：从 $\delta^{(l+1)}$ 推导 $\delta^{(l)}$

已知：$\boldsymbol{z}^{(l+1)} = \boldsymbol{W}^{(l+1)}\boldsymbol{a}^{(l)} + \boldsymbol{b}^{(l+1)}$，$\boldsymbol{a}^{(l)} = f(\boldsymbol{z}^{(l)})$

链式法则逐步展开

$$\begin{aligned}\boldsymbol{\delta}^{(l)} = \frac{\partial L}{\partial \boldsymbol{z}^{(l)}} &= \frac{\partial L}{\partial \boldsymbol{z}^{(l+1)}} \cdot \frac{\partial \boldsymbol{z}^{(l+1)}}{\partial \boldsymbol{a}^{(l)}} \cdot \frac{\partial \boldsymbol{a}^{(l)}}{\partial \boldsymbol{z}^{(l)}} \quad \text{（链式法则）} \\ &= \boldsymbol{\delta}^{(l+1)} \cdot \boldsymbol{W}^{(l+1)} \cdot \mathrm{diag}(f'(\boldsymbol{z}^{(l)})) = (\boldsymbol{W}^{(l+1)})^\top \boldsymbol{\delta}^{(l+1)} \odot f'(\boldsymbol{z}^{(l)})\end{aligned}$$

公式解读

- $(\boldsymbol{W}^{(l+1)})^\top \boldsymbol{\delta}^{(l+1)}$：前向时 $\boldsymbol{W}$ 将信号“散”出去，反向时 $\boldsymbol{W}^\top$ 将误差“收”回来
- $\odot f'(\boldsymbol{z}^{(l)})$：ReLU 时，$z > 0$ 的通道梯度通过，$z \leq 0$ 的通道被**阻断**

两层网络

输入 $x = 2$，目标 $y = 1$，参数：$w_1 = 0.5$，$b_1 = 0.1$，$w_2 = 0.8$，$b_2 = 0.2$
隐藏层：ReLU，输出层：线性，损失：$L = \frac{1}{2}(\hat{y} - y)^2$

前向传播：

$$
\begin{aligned}
z_1 &= w_1 \cdot x + b_1 = 0.5 \times 2 + 0.1 = 1.1 \\
a_1 &= \mathrm{ReLU}(1.1) = 1.1 \\
z_2 &= w_2 \cdot a_1 + b_2 = 0.8 \times 1.1 + 0.2 = 1.08 \\
\hat{y} &= z_2 = 1.08 \\
L &= \frac{1}{2}(1.08 - 1)^2 = 0.0032
\end{aligned}
$$

$$
x = 2 \xrightarrow{w_1,\, b_1} z_1 = 1.1 \xrightarrow{\mathrm{ReLU}} a_1 = 1.1 \xrightarrow{w_2,\, b_2} z_2 = 1.08 \rightarrow L = 0.0032
$$

Step 1：输出层误差 $\delta_2 = \hat{y} - y = 1.08 - 1 = 0.08$

Step 2：输出层参数梯度

$$\frac{\partial L}{\partial w_2} = \delta_2 \cdot a_1 = 0.08 \times 1.1 = 0.088, \quad \frac{\partial L}{\partial b_2} = \delta_2 = 0.08$$

Step 3：误差传回隐藏层 （$z_1 = 1.1 > 0$，$\mathrm{ReLU}' = 1$）

$$\delta_1 = w_2 \cdot \delta_2 \cdot \mathrm{ReLU}'(z_1) = 0.8 \times 0.08 \times 1 = 0.064$$

Step 4：隐藏层参数梯度

$$\frac{\partial L}{\partial w_1} = \delta_1 \cdot x = 0.064 \times 2 = 0.128, \quad \frac{\partial L}{\partial b_1} = \delta_1 = 0.064$$

参数更新（$\eta = 0.1$）

$w_1 \leftarrow 0.4872$，$w_2 \leftarrow 0.7912$。更新后 $\hat{y}' = 1.037$，$L' = 0.00068$，损失**下降 79%**！

梯度消失

每层连乘 $\boldsymbol{W}^\top \cdot f'(\boldsymbol{z})$，若 $\|f'\| < 1$：
例：10 层，每层缩放 0.5：$0.5^{10} = 0.001$
浅层梯度衰减 1000 倍，几乎不更新！

梯度爆炸

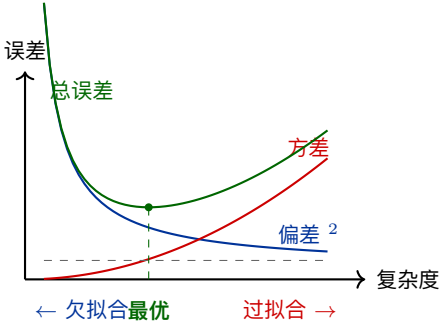
若连乘因子 > 1：$2^{10} = 1024$
梯度膨胀 1024 倍，训练**发散**！

解决方案预告

- ReLU 激活（正区间导数 $= 1$）
- Xavier / He 初始化
- 批归一化（BatchNorm）
- 残差连接（ResNet）
- 梯度裁剪

总误差分解

$$\mathbb{E}[(\hat{y}-y)^2] = \text{Bias}^2 + \text{Var} + \sigma^2_{\text{noise}}$$



权衡

- 太简单 → 高偏差（欠拟合）
- 太复杂 → 高方差（过拟合）
- 深度学习通常面临过拟合——后续讲解正则化

指标	公式	适用场景
MSE / RMSE	$\frac{1}{N}\sum(y_i-\hat{y}_i)^2$	回归任务
MAE	$\frac{1}{N}\sum\lvert y_i-\hat{y}_i\rvert$	回归（有异常值）
Accuracy	$\frac{TP+TN}{\text{All}}$	类别平衡的分类
Precision	$\frac{TP}{TP+FP}$	关注误报（垃圾邮件）
Recall	$\frac{TP}{TP+FN}$	关注漏报（癌症筛查）
F1	$\frac{2\cdot P\cdot R}{P+R}$	平衡 P 和 R
AUC	ROC 曲线下面积	类别不平衡

核心原则

- 类别不平衡时**不要只看准确率**——99% 准确率可能毫无意义
- 漏报代价大 → 优先 Recall；误报代价大 → 优先 Precision
- 训练集上的性能不代表泛化能力——**永远在测试集上评估**

核心知识脉络

1. **线性模型的局限**：决策边界是超平面，无法学习非线性关系
2. **Softmax + 交叉熵**：多分类的标准方案，梯度形式优美 $\frac{\partial L}{\partial \boldsymbol{z}} = \hat{\boldsymbol{y}} - \boldsymbol{y}$
3. **感知机与 XOR**：证明了单层网络的根本局限
4. **多层网络**：隐藏层实现特征空间变换，使非线性问题变线性可分
5. **激活函数**：非线性是深度网络表达能力的关键，ReLU 是默认选择
6. **反向传播**：利用链式法则高效计算所有参数梯度

三个最重要的公式

$$\hat{\boldsymbol{y}} = \mathrm{softmax}(\boldsymbol{z}) \qquad \boldsymbol{\delta}^{(l)} = (\boldsymbol{W}^{(l+1)})^\top \boldsymbol{\delta}^{(l+1)} \odot f'(\boldsymbol{z}^{(l)}) \qquad \frac{\partial L}{\partial \boldsymbol{W}^{(l)}} = \boldsymbol{\delta}^{(l)} (\boldsymbol{a}^{(l-1)})^\top$$

第 4 讲：前馈神经网络深入

- 前馈网络结构详解（深度、宽度、参数量计算）
- 批量矩阵运算与 GPU 并行
- 损失函数详解（MSE / CE / Huber / 为什么分类不用 MSE）
- 反向传播完整算法与维度分析
- 参数初始化策略（Xavier / He）
- PyTorch 自动微分实践

作业

1. 计算 $\boldsymbol{z} = [1.0, 3.0, -1.0, 2.0]$ 的 Softmax 概率分布和交叉熵损失（假设正确类别为第 2 类）
2. 证明：$\sigma'(z) = \sigma(z)(1 - \sigma(z))$，并求 $\sigma'(z)$ 的最大值
3. 给定两层网络 $x = 3, w_1 = 0.4, b_1 = -0.1, w_2 = 0.6, b_2 = 0.3$，ReLU 激活，目标 $y = 2$，MSE 损失。完成前向 + 反向传播，计算所有参数梯度
4. 思考：如果所有激活函数替换为 $f(z) = z$，5 层网络等价于什么？

谢谢！

问题与讨论

孟翰
人工智能学院
中国石油大学（北京）

hmeng@cup.edu.cn